Agente IA Salud — Clase SDC Learning

Clase 90 min · Dictada para SDC Learning

Jack Aguilar — CEO Neuracode

Abril 2026

> ***Clase dictada para SDC Learning*** *· Docente: Jack Aguilar (Neuracode)*

1. Apertura (min 0-5)

Objetivo de la clase

> ***En 90 minutos van a construir un agente IA real, funcional, validable, que hace triage médico.*** *No un ChatGPT wrapper. Un agente con tools, guardrails, structured output y FinOps.*

Qué van a llevarse

- Código Python corriendo con OpenAI Agents SDK (oficial, marzo 2025)
- Arquitectura: 1 agente + 1 guardrail + 3 tools + Pydantic
- Métricas reales de tokens, costo y accuracy
- Repo público como base para su propio producto

Reglas del juego

1. Cuestiona cada decisión (¿realmente lo necesitamos?)
2. Elimina antes de agregar (no RAG, no fine-tuning, no frontend)

3. Simplifica solo lo que sobrevivió
4. Acelera con deadlines agresivos
5. Automatiza solo al final, cuando el proceso manual está validado

---

2. El problema real (min 5-15)

Datos duros

Dato	Fuente	Magnitud
Pacientes que van a emergencia y NO son emergencia	OMS	40-60%
Médicos por habitante en Perú	MINSA	1 / 800 (OMS recomienda 1/500)
Espera promedio en emergencia pública Lima	Defensoría	4-8 horas
Costo consulta privada Lima	Mercado	S/120-200
Costo enfermera triage (salario + carga)	RRHH	~S/50/hora

El Idiot Index

```
Precio de mercado (consulta):  S/150  ≈  $40
Costo real de un triage IA:     $0.00018
Ratio:                          ~222,000x
```

Cuando un ratio precio/costo es absurdo, hay oportunidad de disrupción.

Qué NO estamos haciendo

- **NO** reemplazar al médico
- **NO** dar diagnósticos
- **NO** recetar medicamentos

Qué SÍ estamos haciendo

- **Triage**: clasificar urgencia (ROJO/AMARILLO/VERDE)
- **Derivación**: hospital cercano apropiado
- **Escalamiento humano**: cuando hay duda, médico decide

3. Mercado y negocio (min 15-25)

Empresas que ya validaron el mercado

Empresa	País	Valor
Ada Health	Alemania	13M usuarios · Serie B $90M
Babylon Health	UK	Contrato NHS · IPO $4.2B (2021)
K Health	USA / Israel	Partnership Mayo Clinic
Buoy Health	USA	Spin-off Harvard Medical
Infermedica	Polonia	API B2B · Allianz, PZU

Inversión acumulada global en triage IA: >$500M USD

Modelos de negocio posibles

Modelo	Precio	Cliente objetivo
SaaS B2B clínicas/EPS	S/2,000-10,000/mes por sede	Clínicas medianas, EPS
API pay-per-call	$0.05 por triage	Startups health-tech
White-label aseguradoras	Contrato anual	Pacífico, Rimac, MAPFRE
Comisión derivación	5-10% por receta	Inkafarma, MiFarma

Unit Economics (ver diagrama 03)

- Costo real por triage: **$0.00018** (gpt-4o-mini)
- Precio sugerido: **$0.05**
- Margen: **99.6%**
- Break-even por cliente: ~200 triages/mes (≈ 7/día)

Mercado latinoamericano

- Digital health LatAm 2024: **$13.8B** (Statista)
- CAGR 2024-2030: **19.8%**
- Penetración IA en health LatAm: <5% (oportunidad grande)

---

4. Qué es un Agente (vs un simple LLM) (min 25-35)

Diferencia conceptual

Capacidad	LLM wrapper	
Clasificar	sí	sí
Tool use (APIs, DB)	no	**sí** (MINSA, hospitales, auditoría)
Loop autónomo de decisión	no	**sí** (el LLM decide qué tool llamar)
Guardrails	manual	**nativos** (rechaza fuera de alcance)
Structured output	parcial	**garantizado** (Pydantic)
Tracing/FinOps	manual	**integrado** (SDK)

El loop del agente (ReAct)

```
1. Percibir → recibe síntomas
2. Decidir  → ¿llamo tool? ¿cuál?
3. Actuar   → ejecuta tool (protocolo MINSA, hospital, auditoría)
4. Observar → lee resultado de la tool
5. Repetir  → hasta tener respuesta final estructurada
```

Ver diagrama 01 — Arquitectura del Agente

Stack técnico

- **OpenAI Agents SDK** (`openai-agents` , oficial, marzo 2025)
- **Python 3.10+** con type hints
- **Pydantic 2** para structured output
- **Rich** para CLI bonito
- **pytest + pytest-asyncio** para tests

Qué NO usamos (y por qué)

- **LangChain / CrewAI / LlamaIndex**: conflicto con SDK oficial, más abstracción de la necesaria
- **RAG / vector DB**: protocolos MINSA son pocos y estables, no justifican embeddings
- **Fine-tuning**: prompt + structured output es suficiente para accuracy objetivo
- **Frontend**: la clase prioriza la lógica del agente; UI es trivial después
- **Memoria persistente**: cada triage es independiente

Primero: eliminar. Después, si falta, agregar con datos reales que lo justifiquen.

---

5. Arquitectura (min 35-45)

Flujo de alto nivel

Ver diagrama 02 — Flujo de Triage

```
Síntomas (texto)
    ↓
Safety Guardrail (¿es pedido válido?)
    ↓ sí
Triage Agent Loop (LLM + Tools)
    ├─ consultar_protocolo_minsa
    ├─ buscar_hospital_cercano
    └─ registrar_caso_auditoria
    ↓
Pydantic Schema validation
    ↓
TriageResult (JSON estructurado)
```

Los 3 archivos del agente

```
src/
├── schemas.py   → Pydantic: TriageResult, Hospital, Protocolo (NO LLM)
├── tools.py     → 3 funciones puras + wrapper @function_tool
└── agent.py     → Safety guardrail + Triage agent + run_triage()
```

Más archivos = más complejidad. Si no es necesario, no va.

Structured Output (TriageResult)

```python
class TriageResult(BaseModel):
    nivel: NivelTriage           # ROJO / AMARILLO / VERDE
    confianza: float             # 0.0 - 1.0
    razonamiento: str            # explicación corta
    accion_recomendada: AccionRecomendada
    requiere_medico: bool        # True si confianza < 0.85
    signos_alarma: list[str]     # banderas rojas detectadas
    hospital_sugerido: Hospital | None
    disclaimer: str = DISCLAIMER_OFICIAL  # forzado post-LLM
```

El LLM **no puede alucinar el formato**: Pydantic valida o falla.

Safety Guardrail (InputGuardrail)

Agente pequeño que evalúa el input ANTES de llamar al agente principal.

```
Rechaza:
  - "¿Qué enfermedad tengo?"        (diagnóstico)
  - "¿Qué medicamento tomo?"        (receta)
  - "Interpreta mis análisis"       (labs)
  - "Consejo sobre dieta"           (fuera de alcance)

Acepta:
  - Descripción de síntomas para clasificar urgencia
```

Si rechaza → `tripwire_triggered=True` → corta ejecución, ahorra tokens.

---

6. Zero-Hallucination (min 45-50)

Las 7 técnicas aplicadas (en orden de importancia):

#	Técnica	Cómo
1	**Structured output**	Pydantic `TriageResult` obligatorio — el LLM no puede inventar formato
2	**Protocolos oficiales**	Tool `consultar_protocolo_minsa` con datos validados
3	**Safety guardrail**	Agente separado rechaza pedidos fuera de alcance
4	**Escalamiento humano**	`requiere_medico=True` si confianza < 0.85
5	**Disclaimer forzado**	Se sobreescribe post-LLM con el default de Pydantic
6	**Auditoría obligatoria**	Tool `registrar_caso_auditoria` para compliance
7	**Lista blanca de acciones**	Solo 5 acciones posibles (enum), no texto libre

Principio de precaución

Ante duda entre dos niveles, el agente elige el MÁS URGENTE. Un falso positivo (mandar al hospital a alguien sano) es infinitamente mejor que un falso negativo (mandar a casa a alguien con ACV).

7. FinOps (min 50-55)

Modelo default: gpt-4o-mini (NO cambiar sin razón)

Modelo	Input/1M	Output/1M	Uso
gpt-4o-mini	$0.15	$0.60	**DEFAULT**
gpt-4o	$2.50	$10.00	16x más caro
o3-mini	$1.10	$4.40	7x más caro

Costo desglosado por triage

```
Input tokens:    ~400 (system + síntomas + tool results)
Output tokens:   ~200 (razonamiento + JSON)

Costo = (400 × $0.15 + 200 × $0.60) / 1,000,000
      = (60 + 120) / 1,000,000
      = $0.00018
```

Menos de 2 centésimos de centavo por triage.

Proyección de negocio (ver diagrama 03)

Pacientes/día	Costo/mes	Ingreso/mes @$0.05	Margen
100	$0.54	$150	$149.46
1,000	$5.40	$1,500	$1,494.60
10,000	$54	$15,000	$14,946
100,000	$540	$150,000	$149,460

Costo marginal cercano a cero. Escala sin contratar personal.

Optimizaciones aplicadas

- `temperature=0.1` (no necesitamos creatividad)
- `max_tokens` controlado (no output infinito)
- Guardrail rechaza inputs inválidos ANTES de llamar al agente principal (ahorra tokens)
- Tracing automático del SDK (ver tokens en tiempo real)

---

8. Código en vivo (min 55-75)

Setup

```
git clone https://github.com/jackthony/agente_salud_sdc
cd agente_salud_sdc
pip install -r requirements.txt
cp .env.example .env
# editar .env con OPENAI_API_KEY
```

Demo 1: Un caso real

```
python cli.py
```

Input: *"Hombre 60 años, dolor opresivo en pecho con sudoración, irradia al brazo izquierdo, hace 15 min. Vive en San Isidro."*

Output esperado:

- nivel: **ROJO**
- acción: **LLAMAR_AMBULANCIA**
- confianza: 0.9+
- hospital_sugerido: Clínica Ricardo Palma
- signos_alarma: ["dolor torácico irradiado", "diaforesis", "hombre >50 años"]

Demo 2: Guardrail en acción

Input: *"¿Qué medicamento tomo para la gripe? Dame la dosis."*

Output esperado:

- bloqueado_por_guardrail: **True**
- motivo_bloqueo: "El usuario solicita receta de medicamentos, fuera de alcance"

Demo 3: Suite de 7 casos con métricas

```
python cli.py --casos
```

Muestra tabla con: ID, esperado, predicho, confianza, tokens, costo, latencia, acierto.

Métricas esperadas en clase:

- Accuracy: 85-100% (casos son claros)
- Costo promedio: ~$0.0002/caso
- Latencia: 2-5 segundos

Demo 4: Reto al grupo

> ***"Intenten que se equivoque."***

Los alumnos dictan casos ambiguos. Mostrar:

1. Casos edge donde confianza baja → escalamiento automático
2. Casos fuera de alcance → guardrail bloquea
3. Cuánto costó la clase completa (usualmente < $0.05)

---

9. Tests y validación (min 75-82)

Suite de tests

```
# Unitarios (rápidos, sin gastar tokens)
pytest

# Integración (llama al LLM real, requiere OPENAI_API_KEY)
pytest -m integration
```

Resultado esperado: 33/33 passed

Tipos de tests

Archivo	Qué valida	Costo
test_schemas.py	Pydantic validators, enum values	$0
test_tools.py	Pure functions (protocolo, hospital, auditoría)	$0
test_finops.py	Cálculo de costo USD	$0
test_agent_integration.py	Agente real con guardrails, tools, disclaimer	~$0.005

Cobertura objetivo: 80%+

TDD mandatory: escribir test primero, luego implementación.

10. Cierre (min 82-90)

Qué acabamos de construir

- **Agente real** (no LLM wrapper): tool use + guardrails + structured output
- **FinOps**: centavos por triage, margen 99.6%
- **Zero-hallucination**: 7 técnicas aplicadas en código
- **Validación**: 33 tests, 80%+ cobertura
- **Producción-ready**: falta validación clínica, no validación técnica

Los 5 principios que se aplicaron

Principio	Cómo se vio en clase
Cuestionar	¿Necesitamos RAG? NO. ¿Fine-tuning? NO. ¿Frontend? NO.
Eliminar	3 archivos .py, 1 notebook, 0 sobre-ingeniería
Simplificar	1 agente + 1 guardrail + 3 tools
Acelerar	90 min → agente funcional con métricas
Automatizar al final	Primero validar con médicos reales, después escalar

Próximos pasos si quieren llevarlo a producción

1. **Validación clínica** con 100-500 casos supervisados por médicos reales
2. **Certificación DIGEMID / MINSA** (triage ≠ diagnóstico, menos fricción)
3. **Integración EMR** de clínicas (HL7/FHIR)
4. **Observabilidad** (Langfuse, OpenTelemetry) para tracking de errores
5. **Cultura de feedback loop** con médicos de primera línea

Qué NO hacer

- NO ir a producción sin validación médica
- NO reemplazar personal clínico
- NO bypass al disclaimer ni al guardrail

- NO cambiar a gpt-4o “por si acaso”
- NO meter features que no validaste primero

Apéndice A: Preguntas frecuentes

¿Por qué no usar GPT-4o? 16x más caro, no mejora accuracy en clasificación simple. FinOps gana.

¿Puede reemplazar al médico? NO. Es triage, no diagnóstico. Es filtro, no decisión final.

¿Qué pasa si se equivoca? Escalamiento automático si confianza < 0.85, y siempre principio de precaución (elegir el nivel más urgente).

¿Regulación? Triage no es diagnóstico → menos fricción. Igual se valida con DIGEMID/MINSA antes de producción.

¿Cómo mejorar el agente? Fine-tuning con casos locales, RAG con protocolos hospitalarios, feedback loop con médicos. PERO: antes de eso, validar con datos reales.

¿Cuánto tarda montar esto en una clínica real? - Piloto (1 sede): 2-4 semanas - Validación clínica (100 casos): 1-2 meses - Certificación: 3-6 meses - Go-live: 6-12 meses total

---

Apéndice B: Referencias

- OpenAI Agents SDK docs: https://openai.github.io/openai-agents-python/
- Manchester Triage System (protocolo): https://www.triagenet.net/
- Ada Health business case: https://ada.com/company/
- OMS — emergencias y triage: https://www.who.int/
- Repo de esta clase: https://github.com/jackthony/agente_salud_sdc

---

Contacto

Jack Aguilar — CEO Neuracode

- Web: neuracode.dev
- LinkedIn: linkedin.com/in/jackaguilarc
- TikTok: [@jack.de.neura.code](https://www.tiktok.com/@jack.de.neura.code)
- Instagram: [@neuracode.dev](https://www.instagram.com/neuracode.dev/)
- WhatsApp: +51 982 859 073
- Email: jaaguilar@acity.com.pe

Licencia: MIT · **Disclaimer:** Material educativo. No usar en producción sin validación médica y aprobación regulatoria (DIGEMID / MINSA).